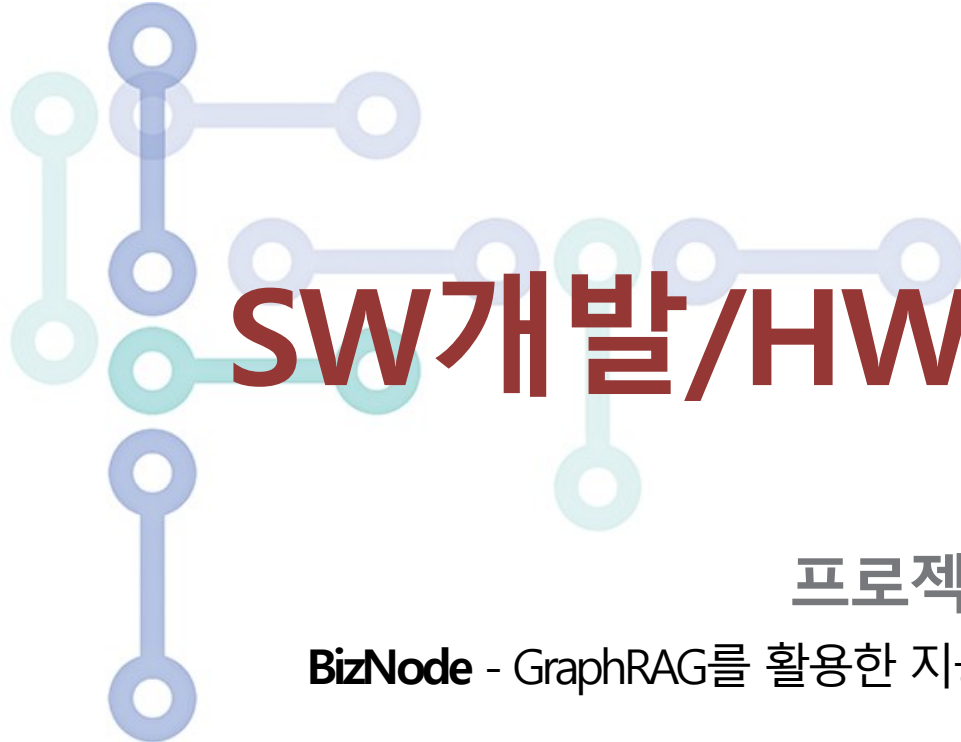




SW개발/HW제작 설계서

프로젝트 명 :

BizNode - GraphRAG를 활용한 지능형 기업 관계 분석 워크스페이스

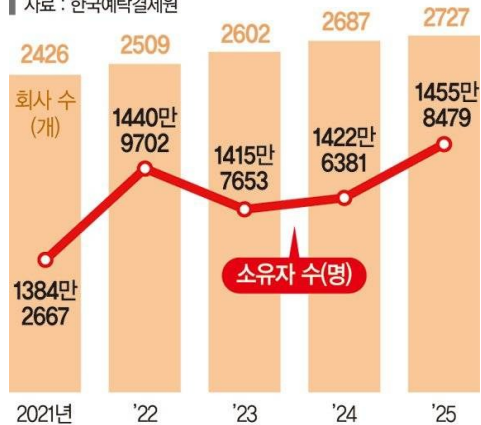
2026. 07. 13

(팀명) 쓱부르스도나스

| 시장/기술 동향 분석

최근 5년간 상장법인 주식 소유자 현황

※ 개인·법인·외국인·기타(실명번호 불명) 대상 작성
자료 : 한국예탁결제원



1. 주식 투자의 대중화와 고도화된 정보 수요 급증

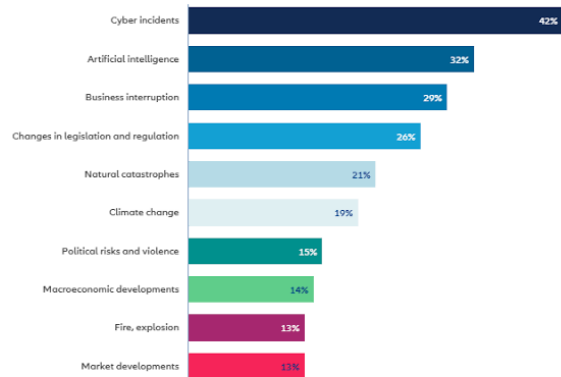
- **투자 대중화:** 주식 소유자 수가 꾸준히 증가하며 2025년 기준 약 1,456만 명에 도달함에 따라, 단순한 재무 지표 조회를 넘어 기업의 펀더멘털과 밸류체인에 대한 심층적인 분석 수요가 폭발적으로 증가하고 있다.
- **정보 탐색의 한계:** 하루 수천 건씩 쏟아지는 DART 공시와 비정형 뉴스 속에서 특정 기업의 파급 효과를 수동으로 분석하는 데에는 막대한 인지적 부하가 발생한다.



The most important business risks in 2026: global

Allianz Risk Barometer 2026

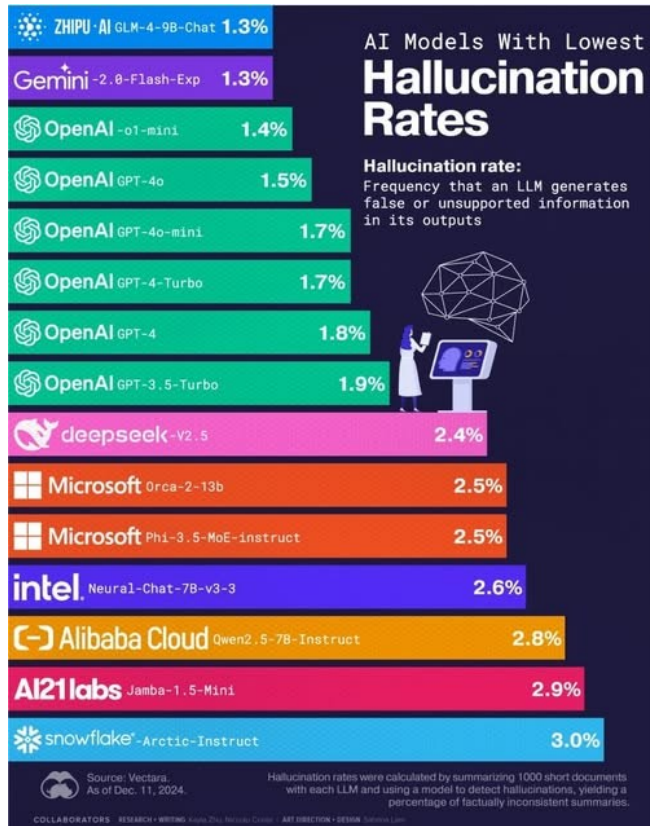
Figures represent the number of risks selected as a percentage of all survey responses from 3,338 respondents. All respondents could select up to three risks per industry, which is why the figures do not add up to 100%.



2. 초연결 비즈니스 생태계와 공급망 연쇄 리스크 심화

- **보이지 않는 리스크:** 현대 비즈니스는 글로벌 공급망으로 얽혀 있으며, 알리안츠 리포트에 따르면 '비즈니스 중단 (공급망 포함)'이 31%로 기업들이 우려하는 리스크 중 하나로 꼽혔다.
- **기존 플랫폼의 구조적 단절:** 네이버 금융, DART 등은 단일 기업의 정보는 잘 제공하지만, 1차 협력사를 넘어 2차, 3차 협력사로 전이되는 복잡한 N차 파급 효과를 직관적으로 파악할 수 없는 구조적 한계가 있다.

| 시장/기술 동향 분석



3. 기존 기술 한계 극복 및 BizNode의 필요성

- **범용 AI의 할루시네이션 극복:** 기존 챗GPT 등 범용 LLM은 금융 도메인에서 가공의 인과관계를 지어낼 위험이 크므로, 팩트(원문)에 기반한 신뢰도 높은 인텔리전스가 필수적이다.
- **솔루션:** 단순 텍스트 기반 검색 환경에서 벗어나, 데이터를 지식 그래프형태로 시각화하고 GraphRAG 기반 다중 홉 추론을 적용하여 숨겨진 공급망 리스크를 가시화하는 'BizNode' 시스템 도입이 필요하다.

| 요구사항 정의서

1. 인증 및 회원 관리

구분	요구사항 ID	요구사항명	설명
S/W	REQ-MEM-01	진입 화면 분기	앱 최초 진입 시 스플래시 화면을 띄운 후, 신규 회원가입 및 기존 회원 로그인을 선택할 수 있는 화면을 제공해야 한다.
	REQ-MEM-02	소셜 연동 로그인	구글, 카카오 등 외부 OAuth API를 활용한 간편 로그인 기능을 제공해야 한다.

2. 검색 및 실시간 데이터 주입

구분	요구사항 ID	요구사항명	설명
S/W	REQ-SCH-01	통합 기업 검색	상장/비상장 기업명 또는 종목 코드를 입력하여 타겟 기업을 검색하고 워크스페이스에 추가할 수 있어야 한다.
	REQ-SCH-02	외부 링크 기반 지식 주입	사용자가 최신 뉴스나 기업 공시 리포트의 URL을 직접 입력할 수 있는 인터페이스를 제공해야 한다.
	REQ-SCH-03	주입 상태 피드백	사용자가 URL을 주입했을 때, 백엔드에서 지식을 추출하고 DB에 병합하는 과정을 로딩 스피너 등으로 시각적으로 안내해야 한다.

| 요구사항 정의서

3. 지식그래프 시각화 및 캔버스

구분	요구사항 ID	요구사항명	설명
S/W	REQ-VIS-01	네트워크 토폴로지 렌더링	Graph DB의 데이터를 바탕으로 중앙 캔버스에 기업(노드)과 비즈니스 관계(엣지)를 시각적인 다이어그램으로 렌더링해야 한다.
	REQ-VIS-02	캔버스 상호작용	사용자가 캔버스를 자유롭게 탐색할 수 있도록 줌인, 줌아웃, 패닝(화면 이동) 기능을 제공해야 한다.
	REQ-VIS-03	노드/엣지 선택 동기화	사용자가 캔버스에서 특정 노드나 엣지를 클릭하면, 좌측 상세 패널에 해당 객체의 상세 정보가 즉시 동기화되어 표시되어야 한다.
	REQ-VIS-04	관계망 유형 필터링	지분 투자, 공급망, 경쟁, 소송 등 사용자가 원하는 특정 비즈니스 관계 유형만 캔버스에 표시하거나 숨길 수 있는 필터링 기능을 제공해야 한다.

| 요구사항 정의서

4. 기업 상세 및 팩트체크 패널

구분	요구사항 ID	요구사항명	설명
S/W	REQ-PNL-01	기업 스냅샷 및 주가 연동	선택된 기업의 개요, AI 요약 정보 및 당일 주가 트렌드를 RDBMS와 연동하여 좌측 패널에 제공해야 한다.
	REQ-PNL-02	핵심 재무 지표 출력	영업이익, 매출액 등 투자 의사결정에 필요한 주요 펀더멘털 수치를 제공해야 한다.
	REQ-PNL-03	공시/뉴스 팩트체크 뷰어	사용자가 캔버스에서 관계(엣지)를 클릭했을 때, 해당 관계를 추론한 근거가 되는 DART 공시 원문 또는 뉴스 기사 원문을 하이라이팅하여 제공해야 한다.

| 요구사항 정의서

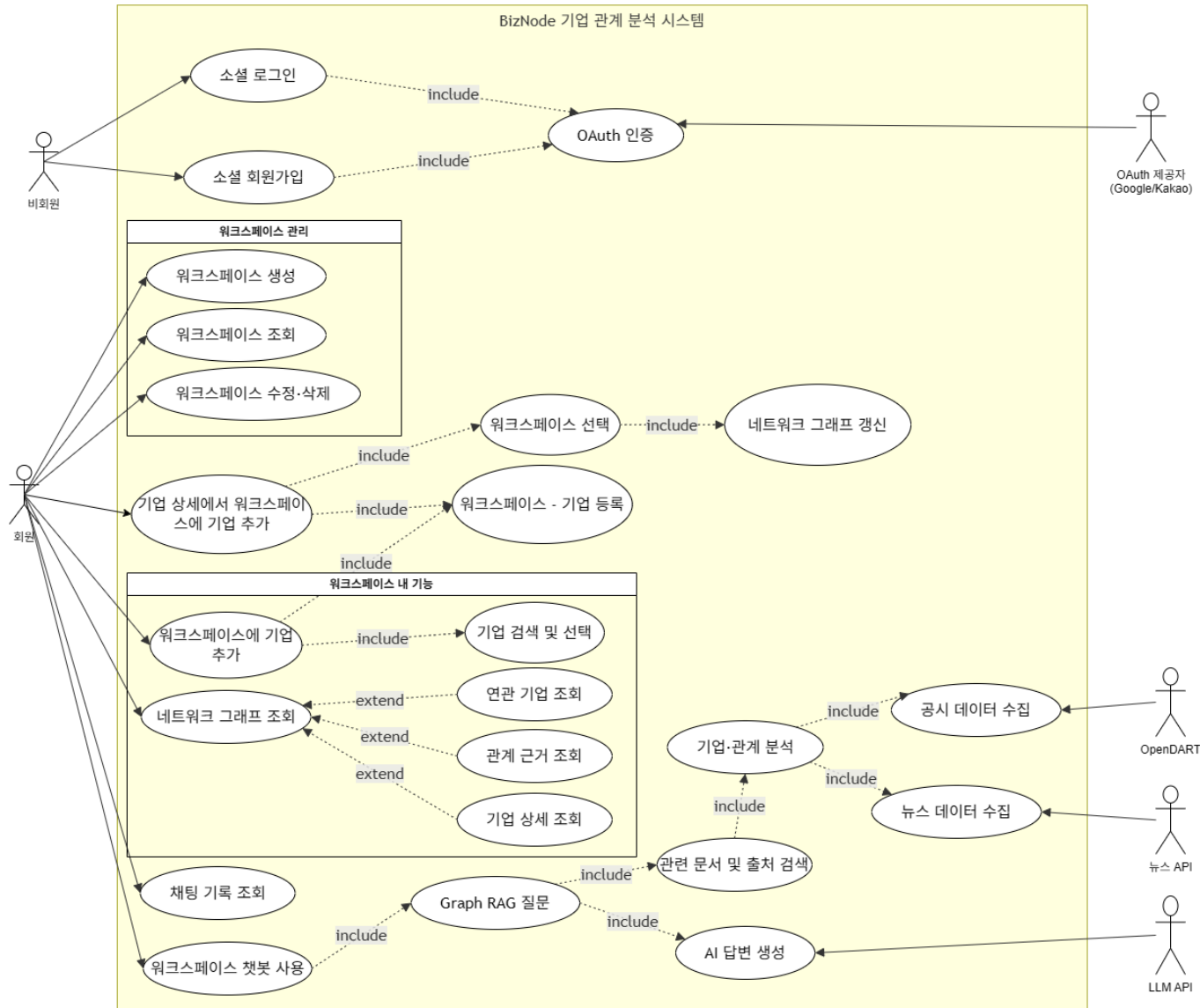
5. GraphRAG AI 챗봇

구분	요구사항 ID	요구사항명	설명
S/W	REQ-BOT-01	자연어 질의 인터페이스	우측 패널에 연쇄 리스크 및 파급 효과를 물어볼 수 있는 자연어 입력 창을 제공해야 한다.
	REQ-BOT-02	심층 추론 및 근거 매핑	멀티 에이전트의 심층 분석 결과를 텍스트로 출력하고, 답변의 기반이 된 팩트체크 출처(Reference)를 매핑하여 명시해야 한다.
	REQ-BOT-03	분석 세션 히스토리	과거에 진행했던 프롬프트 질의 내역과 AI의 응답 세션을 다시 조회할 수 있는 기능을 제공해야 한다.

6. 마이페이지 및 설정

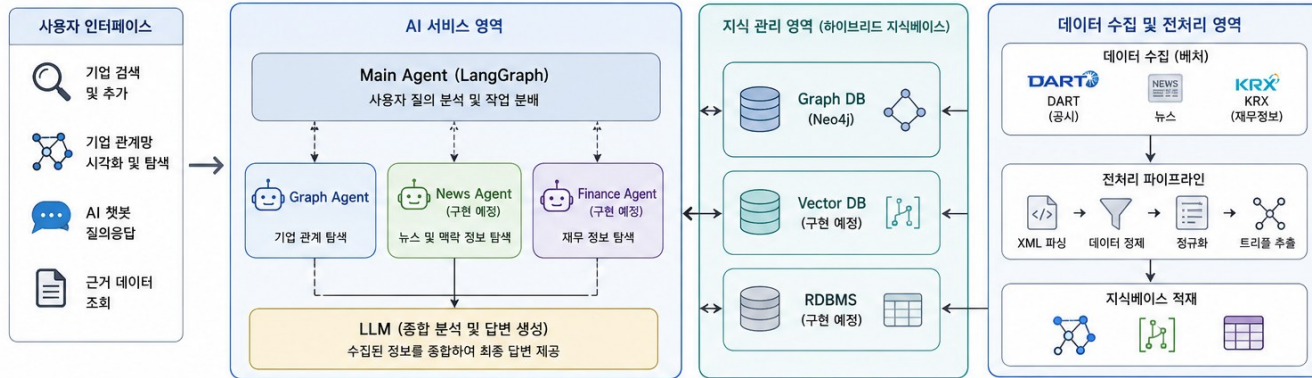
구분	요구사항 ID	요구사항명	설명
S/W	REQ-MYP-01	관심 포트폴리오 관리	지속적인 모니터링이 필요한 기업을 사용자의 관심 포트폴리오에 추가하거나 삭제할 수 있어야 한다.
	REQ-MYP-02	계정 정보 관리	사용자의 비밀번호 변경, 회원 정보 수정 및 플랫폼 회원 탈퇴 기능을 제공해야 한다.

| 유즈케이스 정의서



| 서비스 구성도 - 서비스 시나리오

시스템 구성도 (현재 구현 기준)



백그라운드 데이터 수집 및 적재

① **원천 데이터 수집:** 데이터 파이프라인이 백그라운드에서 Open DART, 네이버 뉴스를 호출하여 기업의 공시, 비정형 뉴스, 정형 재무 데이터를 수집한다.

② **데이터 전처리 및 지식 추출:** 수집된 텍스트를 파싱 및 청킹하고, 사전 정의된 온톨로지 룰에 따라 LLM이 기업 간의 관계(지식 트리플)를 추출한다.

③ **하이브리드 지식베이스 적재:** 추출된 N차 관계는 Graph DB(Neo4j)에, 텍스트 문맥은 Vector DB에, 수치형 마스터 데이터는 RDBMS에 분리 적재한다.

프론트엔드 ↔ 백엔드 상호작용

④ **데이터 요청 (기업 추가):** 사용자가 프론트엔드 UI에서 특정 기업을 검색하여 워크스페이스에 추가를 요청한다.

⑤ **그래프 데이터 전달 (시각화):** 백엔드 (Graph DB)에서 해당 기업과 연관된 밸류체인 데이터를 JSON 형태로 프론트엔드에 전달하여, 화면에 동적인 노드-엣지 다이어그램으로 렌더링한다.

⑥ **근거 데이터 요청 및 반환:** 사용자가 그래프의 엣지(관계)를 클릭하면, 백엔드가 해당 관계가 추출된 공시 원문 및 뉴스 기사 데이터를 Vector DB에서 불러와 좌측 패널에 반환한다.

멀티 에이전트 AI 추론

⑦ **자연어 질의 접수:** 사용자가 우측 패널의 AI 챗봇에 복합 리스크 관련 질의를 입력하여 백엔드로 전달한다.

⑧ **질의 분석 및 다중 탐색:** Main Agent가 질의 의도를 파악한 후, Graph Agent, News Agent, Finance Agent에게 작업을 분배하여 하이브리드 DB의 데이터를 동시다발적으로 조회(조회 쿼리 자동 생성)한다.

⑨ **최종 응답 반환:** 수집된 다차원 컨텍스트를 LLM이 종합적으로 심층 분석한 뒤, 할루시네이션이 차단된 최종 답변 데이터를 프론트엔드 챗봇 UI로 전달한다.

| 서비스 흐름도

서비스 흐름도 (사용자 시나리오)



Step 1. 타겟 기업 탐색 및 워크스페이스 추가

사용자 액션: 분석하고자 하는 핵심 타겟 기업(예: SK하이닉스)을 검색하여 개인 워크스페이스 캔버스에 추가한다.

시스템 처리: 시스템은 백그라운드에서 구축된 하이브리드 지식베이스(DART 공시, 뉴스 API)를 기반으로 해당 기업의 최신 데이터를 연동한다.

Step 2. 기업 관계망 시각화 및 동적 탐색

중앙 (그래프 시각화): 타겟 기업을 중심으로 얹힌 N차 밸류체인 및 연쇄 리스크가 동적인 노드-엣지 지식 그래프로 시각화된다.

좌측 (근거 데이터 탐색): 사용자가 그래프 상의 특정 노드(기업)나 엣지(관계)를 클릭하면, 좌측 패널에 해당 관계가 도출된 명확한 원문 근거(공시, 뉴스) 및 기업 상세 정보가 즉시 호출되어 정보의 신뢰성을 직접 검증할 수 있다.

| 서비스 흐름도

서비스 흐름도 (사용자 시나리오)



Step 3. AI 챗봇 질의응답

우측 (AI 어시스턴트): 사용자는 시각화된 그래프를 탐색하다가 잠재적 리스크를 발견할 경우, 우측 AI 챗봇 패널에 "A사와 우리 회사의 거래 리스크는?"과 같은 복합적인 분석을 자연어로 요청한다.

Step 4. 멀티 에이전트 기반 리스크 심층 분석

작업 분할: Main Agent(LangGraph)가 질의 의도를 파악한 후, Search Agent(맥락 탐색), Finance Agent(재무 탐색)에게 작업을 라우팅한다.

최종 인사이트 제공: 각 에이전트가 하이브리드 DB(Graph, Vector, RDBMS)에서 수집한 다차원 정보를 LLM이 종합하여 할루시네이션이 원천 차단된 최종 심층 분석 결과를 사용자에게 답변합니다.

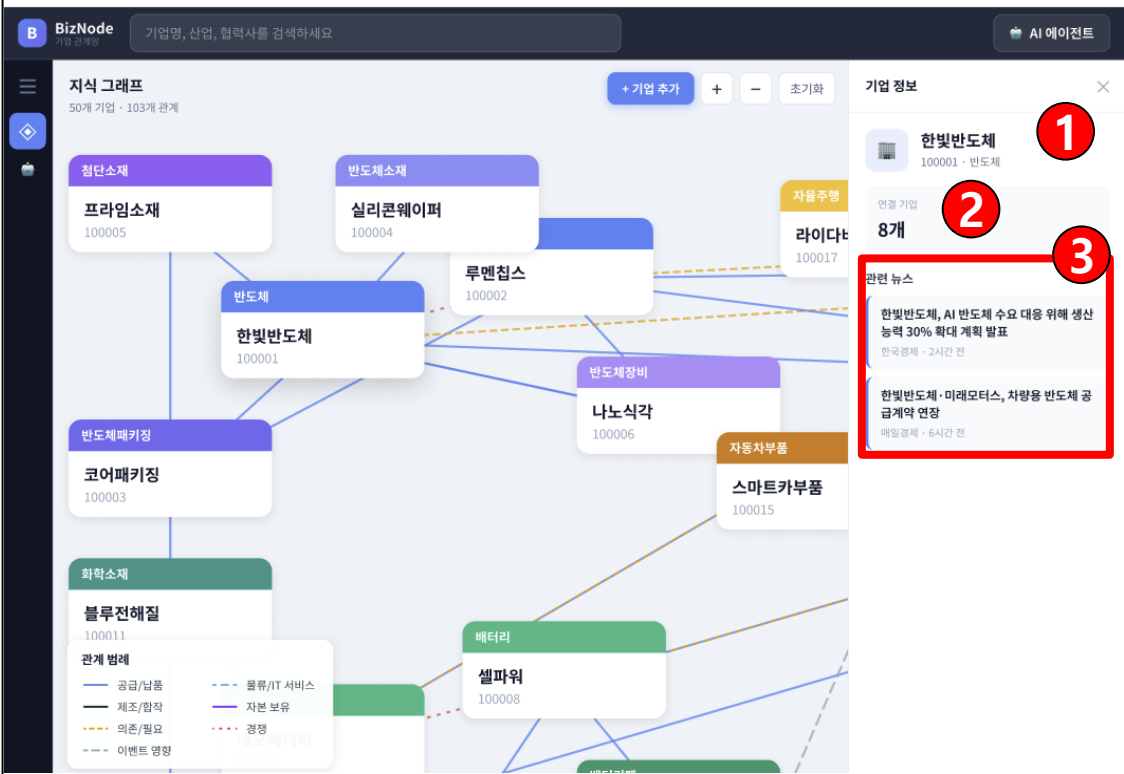
| 메뉴 구성도



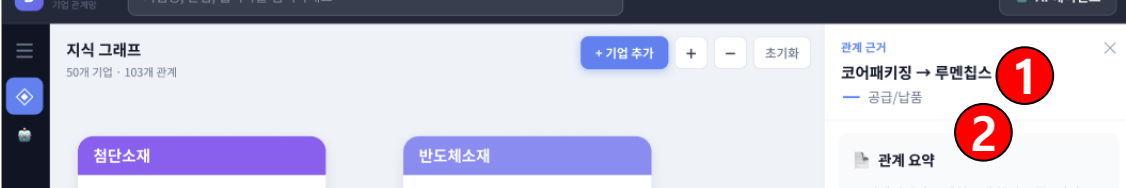
| 화면 설계서

	기능 번호	GRP-01-01
	기능 명	워크스페이스 메인
	기능 설명	서비스의 첫 화면. 제일 좌측 사이드바는 기능 선택, 하위 사이드바로 워크스페이스 선택이 가능하다. 상단 헤더에는 기업 검색이 가능하다.
	처리내용	그래프 캔버스: 기업 관계그래프를 보여주는 캔버스로, 드래그와 확대 및 축소가 가능하다. 노드나 엣지 선택 시 하이라이팅 및 상세 정보 사이드바로 확인 가능하다. 기업 추가 버튼: 원하는 기업을 추가할 수 있다. 확대 축소 버튼: 캔버스 확대, 축소 가능. 초기화 버튼: 워크스페이스 전체를 초기화. 기업 검색창: 기업 검색이 가능하여 워크스페이스 외부 기업의 정보를 확인할 수 있다. AI 챗봇: 우측 사이드에 챗봇 형태로 사용 가능하다.
	비고	연결 기능: 기업 검색 및 추가, 기업 정보 조회, 기업 간 관계 근거 조회, AI 에이전트 챗봇창

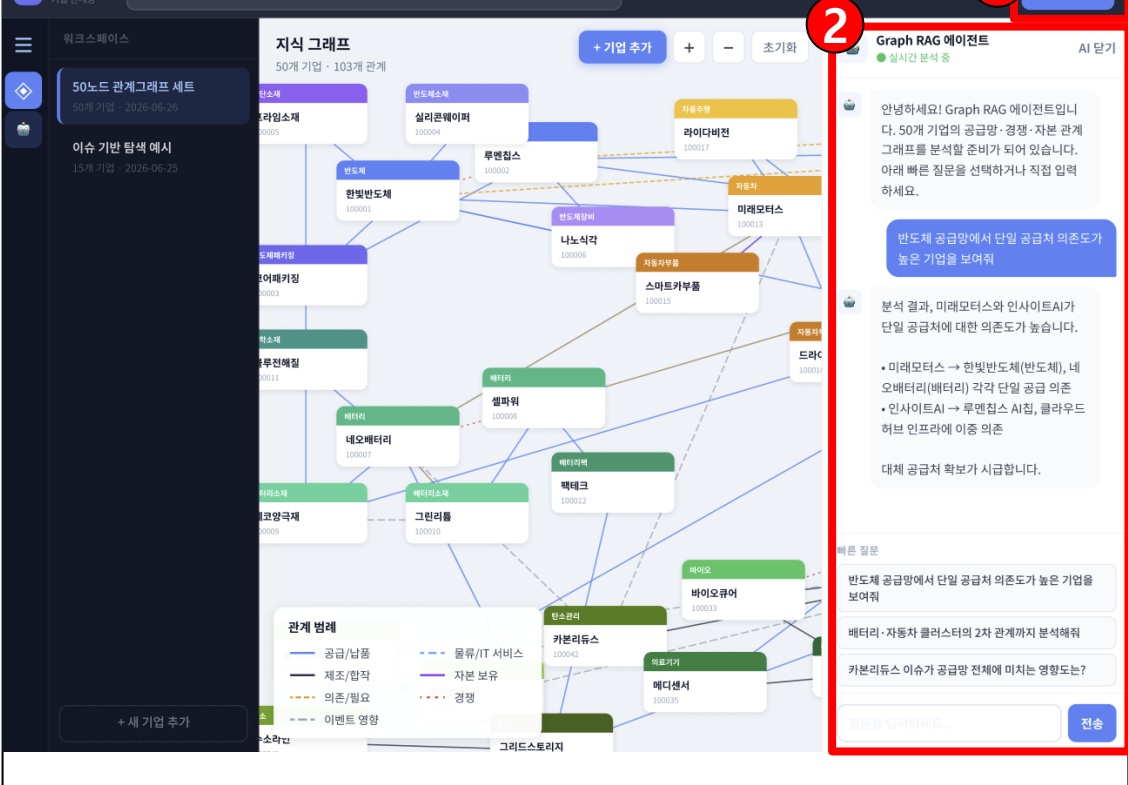
| 화면 설계서

	기능 번호	GRP-01-02
	기능 명	기업 정보 조회
	기능설명	그래프 화면에서 기업 노드를 클릭하면 뜨는 사이드창으로, 기업에 대한 상세 정보를 확인할 수 있다.
	처리내용	기업 개요: 기업명, 종목 코드, 사업 분야 그래프 내 기업 정보: 연관되어 있는 기업 개수와 연관 기업 정보 조회로 바로 라우팅한다. 기업 관련 뉴스: 기업과 관련되어 있는 최신 동향 (뉴스나 공시정보, 증권 리포트 등)을 카드 리스트 형태로 출력한다. 기업 관련 정보(미구현) : 주가, 전일 대비 등락률, 재무 정보, 기업 리스크, 공시 등을 상세히 제공한다.
	비고	

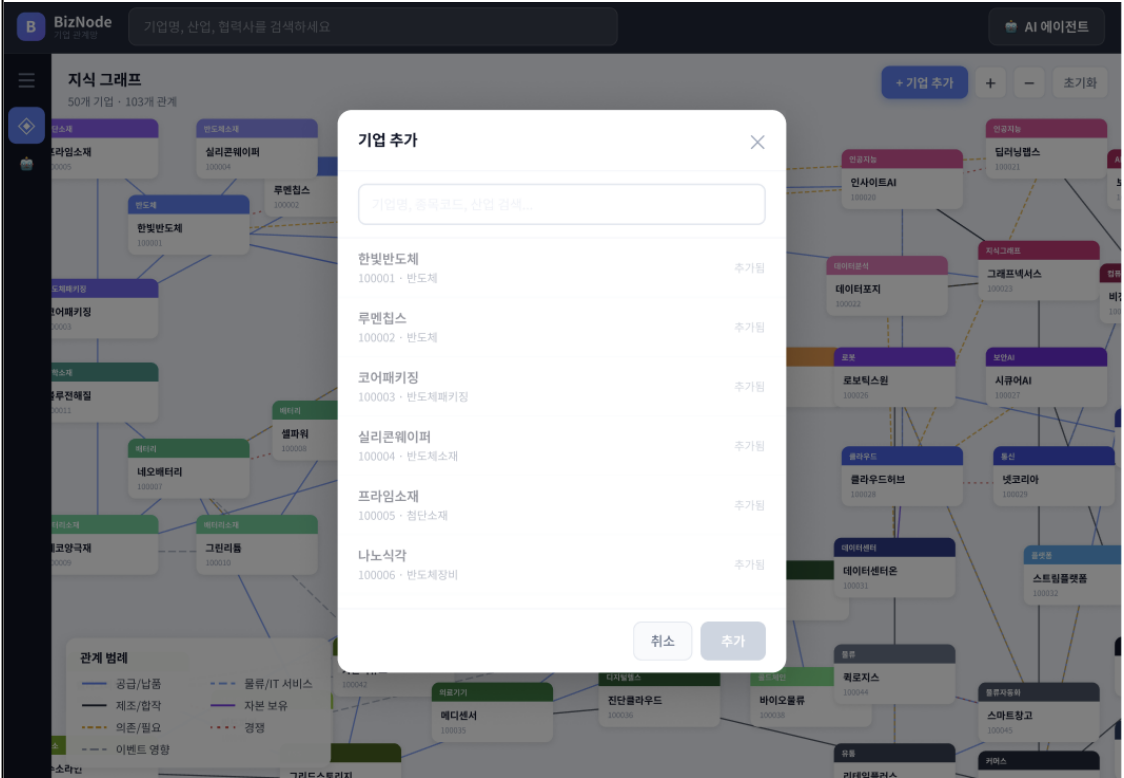
| 화면 설계서

	기능 번호	GRP-01-03
	기능 명	기업 간 관계 근거 조회
	기능설명	그래프에서 기업 간을 연결하는 엣지 클릭 시 띄워지는 사이드창으로, 노드 간의 관계에 대한 부분과 근거를 확인할 수 있다.
	처리내용	1. 연결 기업: 선택한 엣지가 어떤 기업들을 이어주고 있는지 표시한다. 2. 관계 요약: 현재 연결된 노드가 어떠한 관계로 어떻게 이어져 있는지 제공한다. 3. 근거 문서: 관계를 판별한 근거가 있는 문서(리포트, 뉴스 등)를 알려준다. 해당 근거가 접근 가능한 웹페이지일 경우 해당 페이지로 바로 이동 가능하다. 4. 연결 기업 상세 정보: 종목 코드와 기업에 대한 간단한 설명이 출력된다.
	비고	

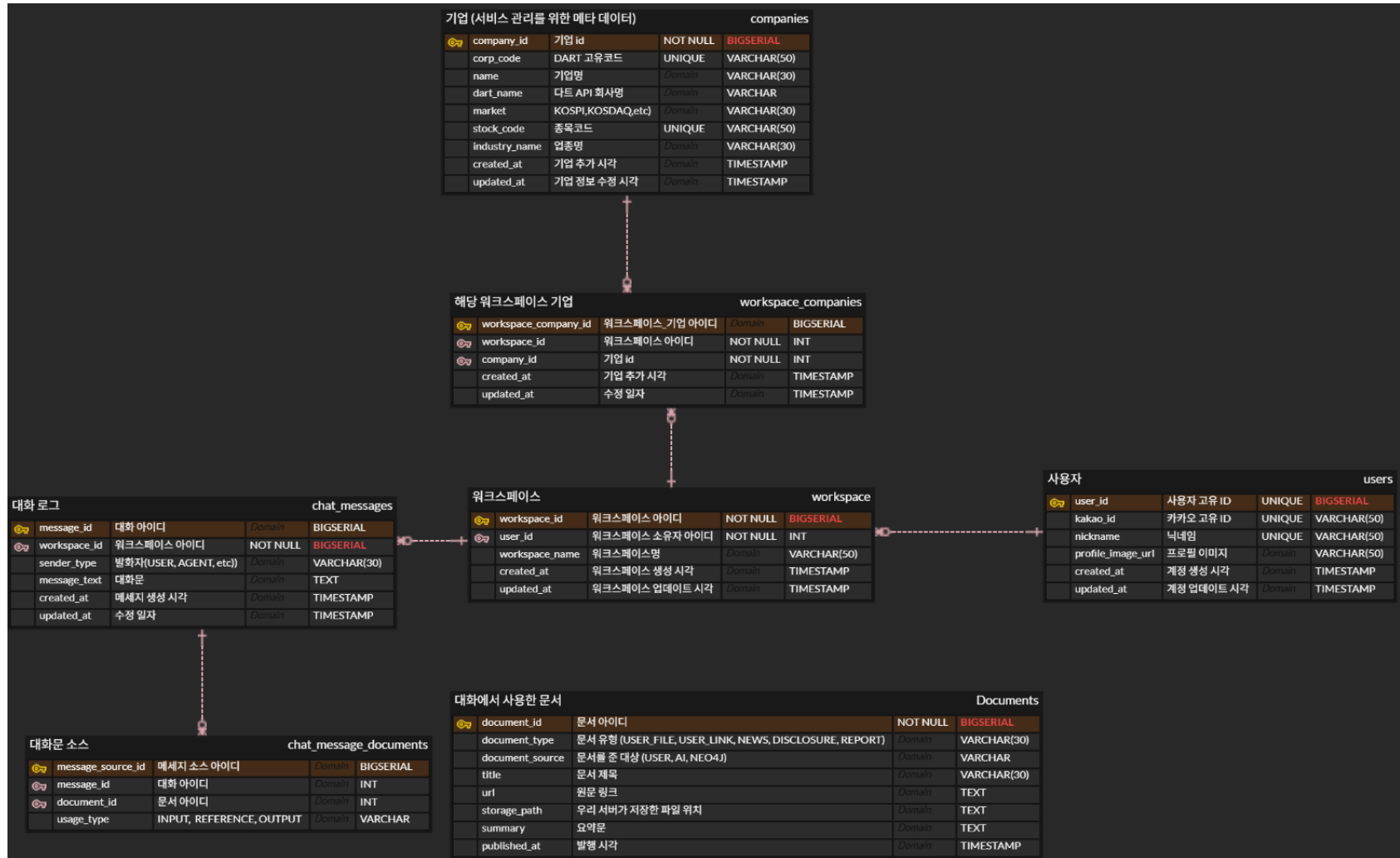
| 화면 설계서

	기능 번호	AI-01-01
	기능 명	AI 에이전트 챗봇창
	기능 설명	그래프에 있는 정보를 기준으로 사용자가 AI 에이전트 챗봇과 대화가 가능하다.
	처리 내용	1. AI 에이전트 챗봇 버튼: AI 에이전트 챗봇을 사용하는 사이드창이 닫히고 열린다. 2. AI 에이전트 챗봇: 채팅창에 질문을 하면 LLM API를 사용하여 그래프에 있는 정보를 기반으로 RAG 방식으로 답변을 제공한다. 답변에 대한 근거가 되는 문서 또한 제공하고 있다.
	비고	

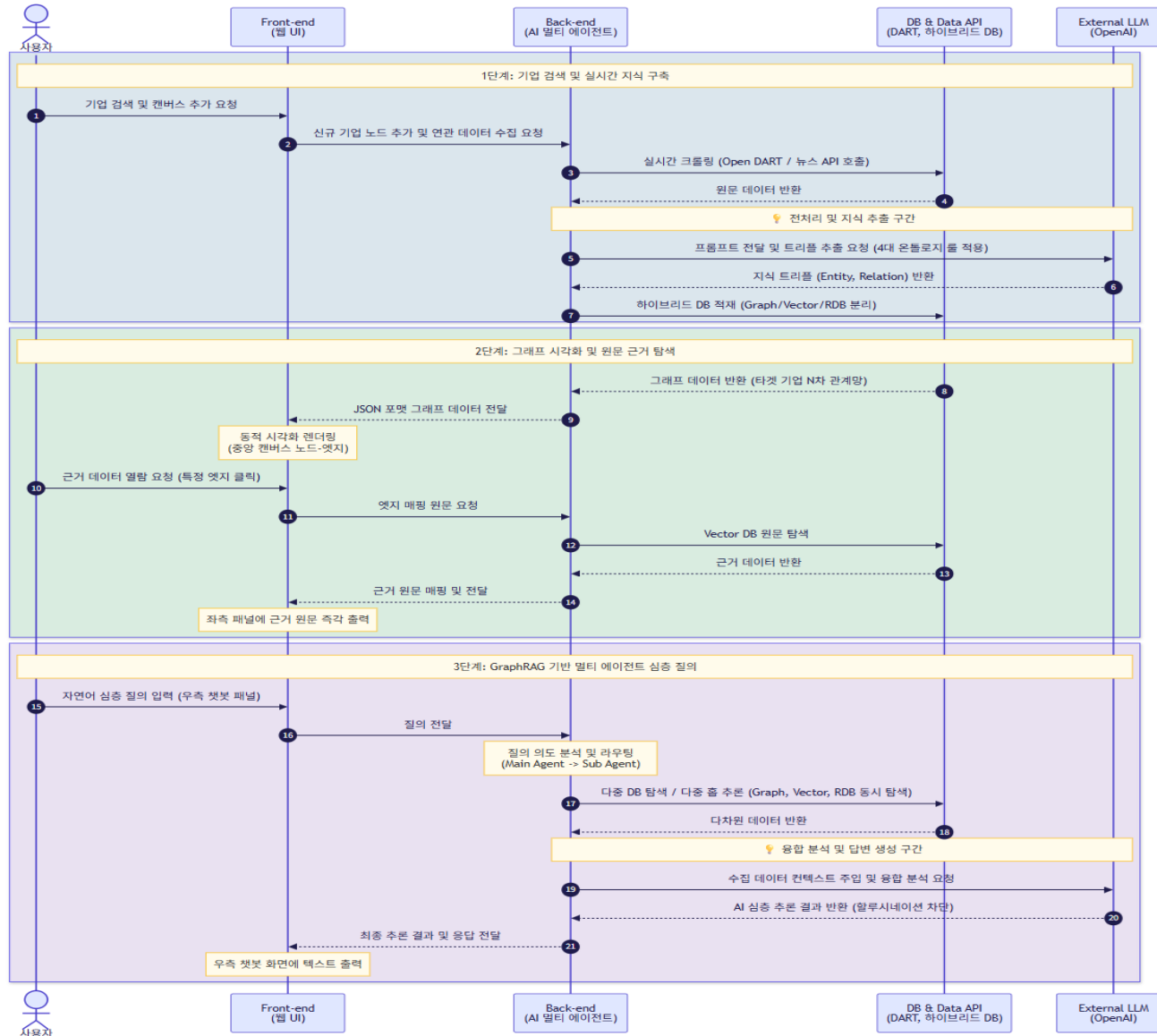
| 화면 설계서

	<table> <tr> <td>기능 번호</td><td>CP-01-01</td></tr> <tr> <td>기능 명</td><td>기업 검색 및 추가</td></tr> <tr> <td>기능설명</td><td>기업 검색이 가능하다. 원하는 기업을 선택한 후 추가 버튼을 누르면 워크스페이스에 기업을 추가할 수 있다.</td></tr> <tr> <td>처리내용</td><td> 기업 검색: 기업명이나 종목코드로 검색이 가능하다. 관련 유사도에 따라 추천 기업을 상단에 노출시킨다. 기업 선택: 원하는 기업을 찾아 내 워크스페이스로 추가하고 싶으면 워크스페이스 선택 모달로 어느 워크스페이스에 넣을 것인지 선택하도록 한다. 이후 추가한 기업에 대한 정보를 읽어와 그래프를 다시 그리는 작업을 거쳐 워크스페이스에 그래프를 재렌더링한다. </td></tr> <tr> <td>비고</td><td></td></tr> </table>	기능 번호	CP-01-01	기능 명	기업 검색 및 추가	기능설명	기업 검색이 가능하다. 원하는 기업을 선택한 후 추가 버튼을 누르면 워크스페이스에 기업을 추가할 수 있다.	처리내용	기업 검색: 기업명이나 종목코드로 검색이 가능하다. 관련 유사도에 따라 추천 기업을 상단에 노출시킨다. 기업 선택: 원하는 기업을 찾아 내 워크스페이스로 추가하고 싶으면 워크스페이스 선택 모달로 어느 워크스페이스에 넣을 것인지 선택하도록 한다. 이후 추가한 기업에 대한 정보를 읽어와 그래프를 다시 그리는 작업을 거쳐 워크스페이스에 그래프를 재렌더링한다.	비고	
기능 번호	CP-01-01										
기능 명	기업 검색 및 추가										
기능설명	기업 검색이 가능하다. 원하는 기업을 선택한 후 추가 버튼을 누르면 워크스페이스에 기업을 추가할 수 있다.										
처리내용	기업 검색: 기업명이나 종목코드로 검색이 가능하다. 관련 유사도에 따라 추천 기업을 상단에 노출시킨다. 기업 선택: 원하는 기업을 찾아 내 워크스페이스로 추가하고 싶으면 워크스페이스 선택 모달로 어느 워크스페이스에 넣을 것인지 선택하도록 한다. 이후 추가한 기업에 대한 정보를 읽어와 그래프를 다시 그리는 작업을 거쳐 워크스페이스에 그래프를 재렌더링한다.										
비고											

| 엔티티 관계도 - ERD

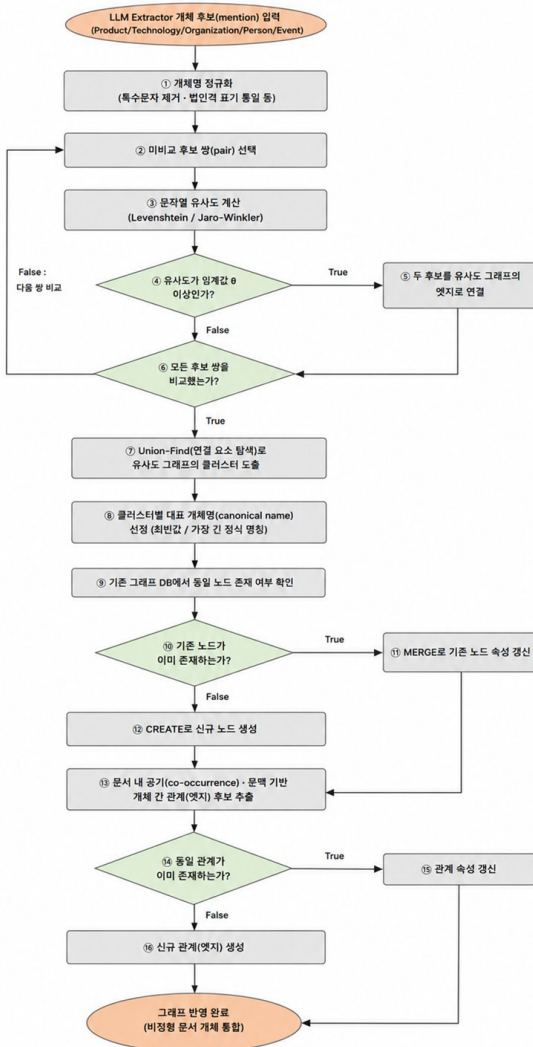


| 기능 처리도(기능 흐름도)



| 알고리즘 명세서

○ 비정형 문서 개체 클러스터링(Entity Resolution) 알고리즘의 흐름도



○ 알고리즘 시나리오 (비정형 문서 개체 클러스터링 / Entity Resolution)

- ① LLM Extractor가 뉴스·사업보고서 등 비정형 문서에서 추출한 개체 후보(Product/Technology/Organization/Person/Event)를 입력받아, 표기 형태를 통일하기 위해 특수문자 제거·법인격 표기(주식회사/㈜ 등) 통일 등 정규화를 수행한다.
- ② 아직 비교하지 않은 개체 후보 쌍(pair)을 하나 선택한다.
- ③ 두 후보의 정규화된 이름에 대해 문자열 유사도(Levenshtein 또는 Jaro-Winkler)를 계산한다.
- ④ 계산된 유사도가 임계값 θ 이상인지 판별한다.
- ⑤ 임계값 이상이면 두 후보를 동일 개체일 가능성이 있는 것으로 보고, 유사도 그래프 상에서 두 후보를 잇는 엣지를 생성한다.
- ⑥ 모든 개체 후보 쌍에 대한 비교가 끝났는지 판별한다. 끝나지 않았다면 ②로 돌아가 다음 쌍을 비교한다.
- ⑦ 모든 쌍의 비교가 끝나면, 유사도 그래프에 대해 Union-Find(또는 BFS/DFS 기반 연결 요소 탐색)를 수행하여 서로 연결된 후보들을 하나의 클러스터로 묶는다.
- ⑧ 각 클러스터에 대해 대표 개체명(canonical name)을 선정한다. (클러스터 내 최빈값 표기, 또는 가장 긴 정식 명칭을 기준으로 선정)
- ⑨ 대표 개체명을 기준으로 기존 그래프 DB(Neo4j)에 동일한 노드가 이미 존재하는지 확인한다.
- ⑩ 기존 노드 존재 여부를 판별한다.
- ⑪ 이미 존재하면 MERGE로 기존 노드의 속성(출처 문서, 언급 횟수 등)을 갱신한다.
- ⑫ 존재하지 않으면 CREATE로 신규 노드를 생성한다.
- ⑬ 동일 문서 내에서 함께 등장(co-occurrence)하거나 문맥상 연관된 개체 간의 관계(엣지) 후보를 추출한다.
- ⑭ 동일한 관계(두 노드 간 동일 관계 유형)가 이미 그래프에 존재하는지 판별한다.
- ⑮ 이미 존재하면 관계의 속성(출처 문서, 최근 언급일 등)만 갱신한다.
- ⑯ 존재하지 않으면 신규 관계(엣지)를 생성한다.

| 알고리즘 상세 설명서

○ 비정형 문서 개체 클러스터링(Entity Resolution) 알고리즘

비정형 문서 개체 클러스터링(Entity Resolution) 알고리즘은 뉴스 기사와 사업보고서 등에서 LLM Extractor가 추출한 개체(Entity)를 정규화하고, 동일한 개체를 식별하여 기존 Knowledge Graph에 통합하기 위한 알고리즘이다. 서로 다른 문서에서 동일한 기업이나 인물, 기술 등이 다양한 표기로 등장하는 문제를 해결하기 위해 문자열 유사도 기반 클러스터링을 수행하며, 이후 그래프 데이터베이스의 노드와 관계를 갱신한다.

먼저 LLM Extractor가 비정형 문서로부터 **Product, Technology, Organization, Person, Event** 등의 개체 후보를 추출한다. 추출된 개체는 특수문자 제거, 법인격 표기(주, 주식회사 등) 통일, 대소문자 및 공백 정리와 같은 전처리를 수행하여 비교 가능한 형태로 정규화한다.

이후 아직 비교하지 않은 개체 후보 쌍(pair)을 하나씩 선택하여 정규화된 이름 간 문자열 유사도를 계산한다. 문자열 유사도는 **Levenshtein Distance** 또는 **Jaro-Winkler** 알고리즘을 이용하여 산출하며, 계산된 유사도가 사전에 정의한 임계값(θ) 이상인 경우 두 개체를 동일 개체 후보로 판단하여 유사도 그래프에 연결한다. 이러한 과정을 모든 개체 후보 쌍에 대해 반복 수행한다.

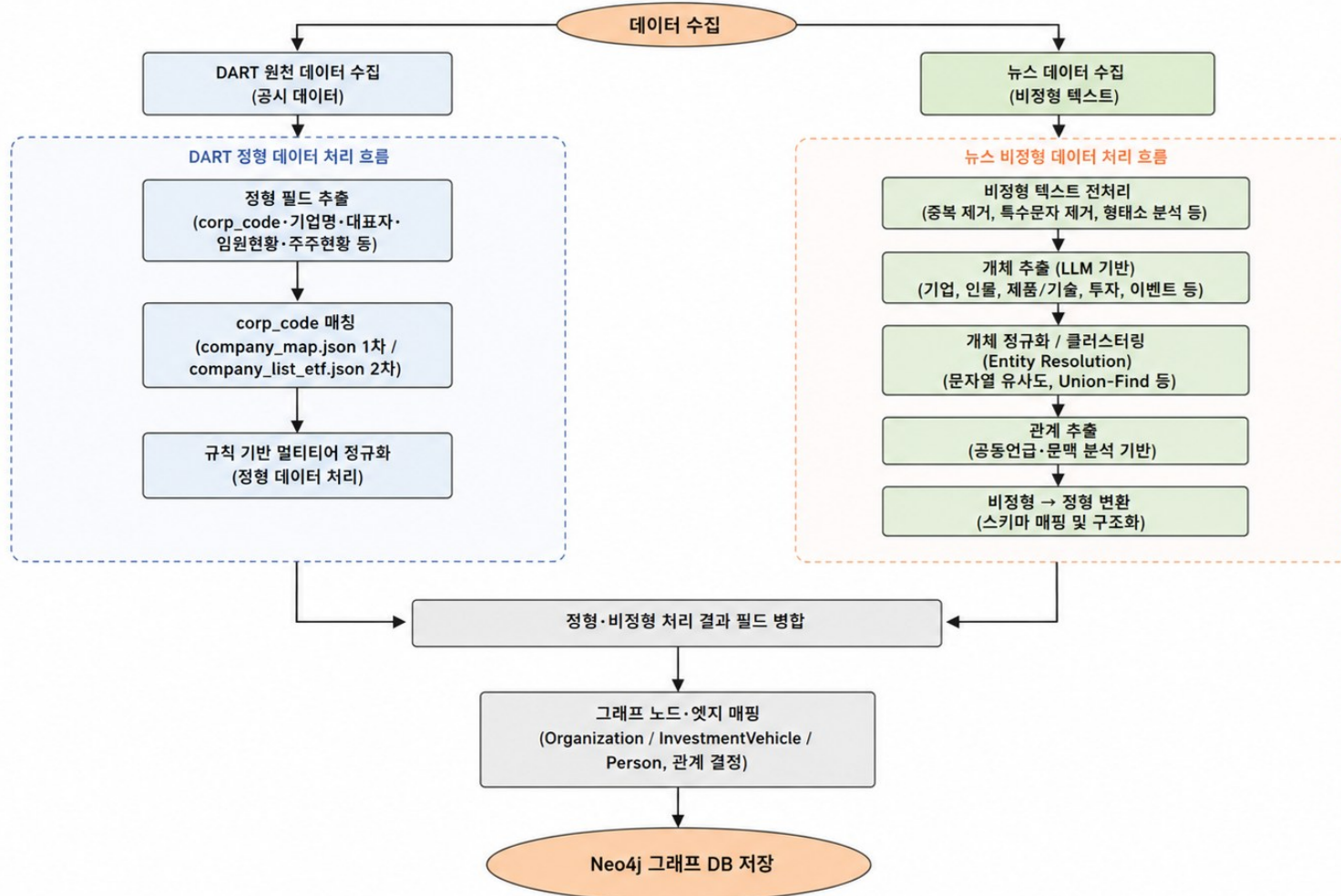
모든 후보 쌍의 비교가 완료되면 **Union-Find(또는 BFS/DFS 기반 연결 요소 탐색)** 알고리즘을 수행하여 서로 연결된 개체 후보들을 하나의 클러스터로 통합한다. 이후 각 클러스터에 대해 가장 많이 사용된 표기 또는 가장 긴 공식 명칭 등을 기준으로 대표 개체명(Canonical Name)을 선정한다.

대표 개체명이 결정되면 Neo4j 그래프 데이터베이스에서 동일한 노드가 이미 존재하는지 확인한다. 기존 노드가 존재하는 경우에는 **MERGE**를 수행하여 출처 문서, 언급 횟수 등의 속성을 갱신하고, 존재하지 않는 경우에는 **CREATE**를 수행하여 새로운 노드를 생성한다.

마지막으로 동일 문서 내 공동 등장(Co-occurrence) 또는 문맥 분석을 기반으로 개체 간 관계 후보를 추출한다. 동일한 관계가 이미 존재하는 경우에는 관계의 출처 문서와 최근 언급일 등의 속성만 갱신하고, 존재하지 않는 경우에는 새로운 관계(Edge)를 생성한다. 이를 통해 비정형 문서에서 추출된 개체와 관계를 기존 Knowledge Graph에 지속적으로 통합 및 갱신하여 최신 기업 관계 정보를 유지하도록 한다.

데이터 수집 처리 정의서

○ BizNode-ETF 데이터 처리 흐름도 (DART 정형 데이터 + 뉴스 비정형 데이터)



| 프로그램 - 목록

기능번호	기능명	설명
LOG-01	인증 및 회원 관리	사용자 식별 및 계정 제어 기능 그룹
LOG-01-01	진입 화면 분기	앱 실행 시 스플래시 화면을 거쳐 신규 가입 또는 기존 회원 로그인 화면으로 이동하는 기능
LOG-01-02	소셜 연동 로그인 (OAuth)	구글, 카카오 등 외부 OAuth API를 활용한 소셜 간편 로그인 및 세션 유지 기능
SCH-01	검색 및 실시간 데이터 주입	타겟 기업 탐색 및 외부 지식 확장 기능 그룹
SCH-01-01	통합 기업 검색	상장/비상장 기업명 또는 종목 코드를 입력하여 캔버스에 추가할 기업을 조회하는 기능
SCH-01-02	외부 링크 기반 지식 주입	사용자가 최신 뉴스나 DART 리포트 URL을 입력하여 백엔드 파이프라인으로 전송하는 인터페이스
SCH-01-03	지식 주입 상태 피드백	백엔드의 정보 추출 및 Graph DB 병합 진행 상황을 로딩 스피너 UI로 안내하는 기능
VIS-01	지식그래프 시각화 및 캔버스	데이터의 시각적 렌더링 및 제어 기능 그룹
VIS-01-01	네트워크 토폴로지 렌더링	백엔드에서 전달받은 JSON 그래프 데이터를 기반으로 캔버스에 노드와 엣지를 시각화하는 기능
VIS-01-02	캔버스 상호작용	그래프 화면 내 마우스/터치 이벤트를 통한 줌인, 줌아웃, 패닝(이동) 기능
VIS-01-03	객체 선택 이벤트 동기화	특정 노드나 엣지 클릭 시 이벤트를 감지하여 좌측 상세 패널로 식별자(ID)를 전달하는 기능
VIS-01-04	관계망 유형 필터링	지분, 공급망, 경쟁 등 특정 관계 속성(Edge Type)을 필터링하여 그래프 뷰를 제어하는 기능

| 프로그램 - 목록

기능 번호	기능 명	설명
PNL-01	기업 상세 및 팩트체크 패널	개별 노드/엣지의 상세 정보 제공 기능 그룹
PNL-01-01	기업 스냅샷 및 주가 연동	선택된 노드(기업)의 개요, AI 요약 텍스트 및 RDBMS와 연동된 당일 주가 정보를 출력하는 기능
PNL-01-02	핵심 재무 지표 출력	선택된 기업의 영업이익, 매출액 등 투자 의사결정에 필요한 주요 수치를 차트나 표로 제공하는 기능
PNL-01-03	공시 및 뉴스 팩트체크 뷰어	선택된 엣지(관계)의 추론 근거가 된 DART 공시 원문 또는 뉴스 텍스트를 호출하고 하이라이팅하는 기능
BOT-01	GraphRAG AI 챗봇	멀티 에이전트 기반 심층 질의응답 기능 그룹
BOT-01-01	자연어 질의 인터페이스	사용자로부터 리스크 분석, 파급 효과 등 복합적인 자연어 질의를 입력받는 채팅 UI
BOT-01-02	심층 추론 및 근거 매핑 출력	멀티 에이전트가 반환한 심층 분석 텍스트와 해당 답변의 출처를 매핑하여 렌더링하는 기능
BOT-01-03	분석 세션 히스토리 조회	과거 질의 프롬프트 내역과 AI 응답 세션 기록을 DB에서 불러와 재조회할 수 있는 기능
MYP-01	마이페이지 및 설정	개인화 데이터 및 계정 관리 기능 그룹
MYP-01-01	관심 포트폴리오 관리	사용자가 북마크한 관심 기업 목록을 조회, 추가, 삭제할 수 있는 관리 기능
MYP-01-02	계정 정보 관리	사용자의 비밀번호 변경, 프로필 수정 및 서비스 회원 탈퇴 처리를 수행하는 기능

| 테이블 정의서 - ERD

users (사용자 정보)

논리명	물리명	타입	길이	PK	FK	NULL	UNIQUE	DEFAULT	설명
사용자 고유 ID	user_id	BIGSERIAL	-	O	-	N	O	AUTO	사용자 PK
카카오 고유 ID	kakao_id	VARCHAR	50	-	-	N	O	-	카카오 로그인 ID
닉네임	nickname	VARCHAR	50	-	-	N	O	-	사용자 닉네임
프로필 이미지	profile_image_url	VARCHAR	50	-	-	Y	-	NULL	프로필 이미지 URL
생성일	created_at	TIMESTAMP	-	-	-	N	-	CURRENT_TIMESTAMP	계정 생성 시간
수정일	updated_at	TIMESTAMP	-	-	-	N	-	CURRENT_TIMESTAMP	계정 수정 시간

workspace (워크스페이스)

논리명	물리명	타입	길이	PK	FK	NULL	DEFAULT	설명
워크스페이스 ID	workspace_id	BIGSERIAL	-	O	-	N	AUTO	워크스페이스 PK
사용자 ID	user_id	INT	-	-	users.user_id	N	-	소유 사용자
워크스페이스명	workspace_name	VARCHAR	50	-	-	N	-	워크스페이스 이름
생성일	created_at	TIMESTAMP	-	-	-	N	CURRENT_TIMESTAMP	생성 시간
수정일	updated_at	TIMESTAMP	-	-	-	N	CURRENT_TIMESTAMP	수정 시간
수정일	updated_at	TIMESTAMP	-	-	-	N	-	CURRENT_TIMESTAMP

| 테이블 정의서 - ERD

companies (서비스 관리를 위한 메타 데이터)

논리명	물리명	타입	길이	PK	FK	NULL	UNIQUE	DEFAULT	설명
기업 ID	company_id	BIGSERIAL	-	O	-	N	O	AUTO	기업 PK
DART 기업코드	corp_code	VARCHAR	50	-	-	N	O	-	DART 기업 고유 코드
기업명	name	VARCHAR	30	-	-	N	-	-	서비스 기업명
DART 회사명	dart_name	VARCHAR	-	-	-	Y	-	NULL	공시상 회사명
시장	market	VARCHAR	30	-	-	Y	-	NULL	KOSPI/KOSDAQ
종목코드	stock_code	VARCHAR	50	-	-	Y	O	NULL	상장 종목코드
업종명	industry_name	VARCHAR	30	-	-	Y	-	NULL	업종
생성일	created_at	TIMESTAMP	-	-	-	N	CURRENT_TIMESTAMP	생성 시간	
수정일	updated_at	TIMESTAMP	-	-	-	N	CURRENT_TIMESTAMP	수정 시간	

| 테이블 정의서 - ERD

workspace_companies (해당 워크스페이스에 있는 기업)

논리명	물리명	타입	PK	FK	NULL	DEFAULT	설명
워크스페이스 기업 ID	workspace_company_id	BIGSERIAL	O	-	N	AUTO	PK
워크스페이스 ID	workspace_id	INT	-	workspace.workspace_id	N	-	워크스페이스
기업 ID	company_id	INT	-	companies.company_id	N	-	등록 기업
생성일	created_at	TIMESTAMP	-	-	N	CURRENT_TIMESTAMP	추가 시간
수정일	updated_at	TIMESTAMP	-	-	N	CURRENT_TIMESTAMP	수정 시간
수정일	updated_at	TIMESTAMP	-	-	-	N	-

chat_messages (대화 로그 테이블)

논리명	물리명	타입	PK	FK	NULL	DEFAULT	설명
메시지 ID	message_id	BIGSERIAL	O	-	N	AUTO	메시지 PK
워크스페이스 ID	workspace_id	BIGSERIAL	-	workspace.workspace_id	N	-	소속 워크스페이스
발신자 타입	sender_type	VARCHAR	-	-	N	-	USER / AI / AGENT
메시지	message_text	TEXT	-	-	N	-	대화 내용
생성일	created_at	TIMESTAMP	-	-	N	CURRENT_TIMESTAMP	생성 시간
수정일	updated_at	TIMESTAMP	-	-	N	CURRENT_TIMESTAMP	수정 시간

| 핵심 소스코드(1)

OpenDart 원본 보고서 정제 파이프라인

```
from schemas.company_report import CompanyReport
from schemas.source import Source

BUSINESS_SECTION_NAME = "사업의 내용"

You, 17 hours ago | 1 author (You)
class ReportDownloader(Protocol):
    """접수번호로 본문 XML을 확보해 로컬 경로를 반환하는 컴포넌트의 인터페이스."""
    def __call__(self, rcept_no: str) -> Path: ... "rcept": Unknown word.

You, 17 hours ago | 1 author (You)
class SectionParser(Protocol):
    """XML 파일을 '{섹션명: 원문}' 으로 구조화하는 컴포넌트의 인터페이스."""
    def __call__(self, xml_path: Path) -> dict[str, str]: ...

You, 17 hours ago | 1 author (You)
class TextCleaner(Protocol):
    """섹션 원문을 정제된 텍스트로 바꾸는 컴포넌트의 인터페이스."""
    def __call__(self, raw_text: str) -> str: ...

def extract_company_report(
    company_id: str, # 회사 식별자
    rcept_no: str, # OpenDART 접수번호 "rcept": Unknown word.
    report_year: str, # 사업연도
    report_type: str, # 보고서 종류
    *, # 키워드 전용 인자
    download: ReportDownloader = dart_downloader.download_and_extract, # XML 확보 컴포넌트.
    parse_sections: SectionParser = dart_xml_parser.parse_sections, # XML -> 섹션 구조화 컴포넌트.
    clean_text: TextCleaner = dart_text_cleaner.clean_text, # 섹션 원문 -> 정제 텍스트 컴포넌트.
    section_name: str = BUSINESS_SECTION_NAME, # 추출할 섹션명
) -> Optional[CompanyReport]:

    xml_path = download(rcept_no) "rcept": Unknown word.
    sections = parse_sections(xml_path)

    raw_section_text = sections.get(section_name)
    if raw_section_text is None:
        return None

    cleaned_text = clean_text(raw_section_text)
    if not cleaned_text:
        return None

    return CompanyReport(
        company_id=company_id,
        rcept_no=rcept_no, "rcept": Unknown word.
        report_year=report_year,
        report_type=report_type,
        section_name=section_name,
        text=cleaned_text,
        source=Source(dart_rcept_no=rcept_no, xml_path=str(xml_path)), "rcept": Unknown word.
    )

You, 17 hours ago * feat: OpenDART ETL 파이프라인 초기 구현
```

OpenDart API 원본 데이터 정규화 파이프라인 일부 코드

```
def normalize_investments(rows: list[dict[str, Any]], corp_code: str) -> NormalizedDocument:
    """타입인 출자현황을 Company/InvestmentVehicle 엔티티와 INVESTS_IN 관계로 변환한다."""
    entities: dict[tuple[EntityType, str], EntityDTO] = {}
    relationships: list[RelationshipDTO] = []

    for row in rows:
        name = clean_name(row.get("inv_prm"))
        if name is None or is_total_row(name):
            continue

        total_assets = parse_float(row.get("recent_bsns_year_fnnr_sttus_tot_assets")) "bsns": Unknown word.
        net_profit = parse_float(row.get("recent_bsns_year_fnnr_sttus_thstrm_ntpf")) "bsns": Unknown word.

        # 피투자 회사를 Company 또는 InvestmentVehicle로 구분한다.
        vehicle_type = is_investment_vehicle(name)
        entity_type = EntityType = "InvestmentVehicle" if vehicle_type else "Company"
        entity_key = (entity_type, name)

        if entity_key not in entities:
            if vehicle_type:
                vehicle = InvestmentVehicleDTO(name=name, vehicle_type=vehicle_type)
                properties = vehicle.to_properties()
            else:
                matched_corp_code, matched_stock_code, matched_market = resolve_investee_corp_code(name) "inve
                company = CompanyDTO(
                    name=name,
                    corp_code=matched_corp_code,
                    stock_code=matched_stock_code,
                    market=matched_market,
                    total_assets=total_assets,
                    net_profit=net_profit,
                )
                properties = company.to_properties()

            entities[entity_key] = EntityDTO(type=entity_type, key=name, properties=properties)

        # 기초·기말 지분율을 비교해 증감 정보를 계산한다.
        share_ratio = parse_float(row.get("trmnd_blsce_qota_rt")) "trmnd": Unknown word.
        previous_share_ratio = parse_float(row.get("bsis_blsce_qota_rt")) "bsis": Unknown word.
        share_ratio_change = (
            round(share_ratio - previous_share_ratio, 2)
            if share_ratio is not None and previous_share_ratio is not None
            else None
        )
        investment_increased = (
            share_ratio > previous_share_ratio
            if share_ratio is not None and previous_share_ratio is not None
            else False
        )
        investment_decreased = (
            share_ratio < previous_share_ratio
            if share_ratio is not None and previous_share_ratio is not None
            else False
        )

        # INVESTS_IN 관계를 생성한다.
```

| 핵심 소스코드(2)

OpenDart xml 파일 핵심 데이터 추출 코드

```
extract_company_report.py > ...

if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")
    sys.stderr.reconfigure(encoding="utf-8")

DEFAULT_REPORT_TYPE = "사업보고서"

BASE_DIR = os.path.dirname(os.path.dirname(__file__))
OUTPUT_DIR = os.path.join(BASE_DIR, "data", "company_reports")

def parse_args(argv: Optional[list[str]] = None) -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--company-id", required=True, help="회사 식별자(예: 종목코드 005930)")
    parser.add_argument("--rcept-no", required=True, help="OpenDART 접수번호(예: 20250318000763)")
    parser.add_argument(
        "--report-year",
        help="사업연도. 생략하면 접수번호(rcept-no) 앞 4자리(접수연도)를 사용한다.",
        "rcept": "Unknown word."
    )
    parser.add_argument(
        "--report-type",
        default=DEFAULT_REPORT_TYPE,
        help="보고서 종류(기본값: {DEFAULT_REPORT_TYPE})",
    )
    return parser.parse_args(argv)

# 사업보고서 DTO를 JSON으로 직렬화하여 파일로 저장함.
def save_report_json(report: CompanyReport) -> str:
    os.makedirs(OUTPUT_DIR, exist_ok=True)
    file_path = os.path.join(
        OUTPUT_DIR, f"{report.company_id}_{report.report_year}_business_report.json"
    )

    payload = {
        "company_id": report.company_id,
        "report_year": int(report.report_year),
        "report_type": report.report_type,
        "section": report.section,
        "text_length": len(report.text),
        "text": report.text,
        "source": {
            "dart_rcept_no": report.source.dart_rcept_no,
            "xml_path": report.source.xml_path,
        },
    }

    with open(file_path, "w", encoding="utf-8") as f:
        json.dump(payload, f, ensure_ascii=False, indent=2)

    return file_path

def main() -> None:
    args = parse_args()
    report_year = args.report_year or args.rcept_no[:4]
    "rcept": "Unknown word."

    report = extract_company_report(
        company_id=args.company_id,
```

추출된 회사를 원본 데이터 매핑, 정제 코드

```
# 주어진 디렉토리에서 API_PIPELINE에 정의된 API의 raw 파일이 있는 corp_code 목록을 찾음.
def discover_corp_codes() -> list[str]:
    corp_codes: set[str] = set()
    if not os.path.isdir(RAW_DIR):
        return []
    for filename in os.listdir(RAW_DIR):
        match = _RAW_FILENAME_RE.match(filename)
        if match and match.group("api_name") in API_PIPELINE:
            corp_codes.add(match.group("corp_code"))
    return sorted(corp_codes)

def load_raw_rows(corp_code: str, api_name: str) -> list[dict[str, Any]]:
    file_path = os.path.join(RAW_DIR, f"{corp_code}_{api_name}.json")
    with open(file_path, encoding="utf-8") as f:
        data = json.load(f)
    return data["list"]

# 주어진 corp_code와 api_name에 해당하는 raw JSON을 읽고, normalizer를 적용하고, validator를 적용한 뒤 normalized JSON을
def process_one(corp_code: str, api_name: str) -> None:
    normalize_fn, validate_fn = API_PIPELINE[api_name]

    try:
        rows = load_raw_rows(corp_code, api_name)
    except FileNotFoundError:
        print(f"{corp_code} {api_name}: raw 파일 없음, 건너뛸")
        return
    except (json.JSONDecodeError, KeyError) as exc:
        print(f"{corp_code} {api_name}: raw 파일 파싱 실패({exc!r}), 건너뛸")
        return

    try:
        document = normalize_fn(rows, corp_code)
        validated, report = validate_fn(document)
    except Exception as exc:
        print(f"{corp_code} {api_name}: 정규화/검증 실패({exc!r}), 건너뛸")
        return

    out_dict = validated.to_dict()
    out_dict["entities"] = enrich_company_entities(out_dict["entities"])

    os.makedirs(NORMALIZED_DIR, exist_ok=True)
    out_path = os.path.join(NORMALIZED_DIR, f"{corp_code}_{api_name}.json")
    with open(out_path, "w", encoding="utf-8") as f:
        json.dump(out_dict, f, ensure_ascii=False, indent=2)

    print(
        f"{corp_code} {api_name}: {len(document.relationships)}건 처리, "
        f"{len(report.dropped)}건 드롭, {len(report.warned)}건 경고"
    )

def run(corp_codes: list[str]) -> None:
    for corp_code in corp_codes:
        for api_name in API_PIPELINE:
            process_one(corp_code, api_name)
```

| 참조-개발 환경 및 설명

구분		상세내용
S/W 개발환경	OS	Windows 11
	개발환경(IDE)	VScode, intelliJ, Antigravity
	개발도구	Git, GitHub, Docker Desktop, Claude Code
	개발언어	Python, Java, TypeScript, React
	기타사항	Neo4j, ChromaDB, OpenDART Open API, Taiwind CSS, Vite, D3.js
프로젝트 관리환경	형상관리	Git, GitHub
	의사소통관리	Discord, Notion, Kakao Talk, Webex
	기타사항	

Thank you

